Innovation partner H2S HACK2SKILL

Media partner YOURSTORY

AI for Bharat Hackathon

Powered by aws

Team Name : Phantom Troup

Team Leader Name : Seetharam Killivalavan

Problem Statement : AI for Learning & Developer Productivity

FragilityGraph

Real-time Structural Debt Prediction using Graph Neural Networks

Predicting code breaks before they happen

The Problem: The Ripple Effect

Problem Statement: Modern software development is plagued by "The Ripple Effect" where code changes in one area silently break functionality in another due to hidden dependencies that standard tools cannot detect.

The Gap:

- Current tools are reactive - they tell you something is broken AFTER build failures or production bugs
- Static analysis (linters/compilers) cannot see implicit dependencies
- Changes in Service A can cause silent logic failures in Module Z

The Cost:

- "Structural Debt" makes developers afraid to refactor
- This "fear-driven development" slows feature velocity by up to 40% in mature projects
- Teams spend excessive time debugging ripple effects instead of building features

Our Solution: FragilityGraph (Real-time Structural Debt Prediction using Graph Neural Networks)

How It Solves the Problem:

1. TOPOLOGICAL FORECASTING
   - Treats code as a multi-layered directed graph instead of text
   - Analyzes structural relationships between components
   - Maps the entire codebase dependency network

2. GNN-POWERED PREDICTION
   - Uses Graph Neural Networks to learn code "vibration" patterns
   - Simulates how changes ripple through the dependency graph
   - Similar to how bridge engineers predict structural stress

3. TEMPORAL LEARNING
   - Analyzes Git history to identify files that historically "break together"
   - Learns co-change patterns even when files aren't explicitly linked
   - Builds predictive models from real-world failure patterns

4. REAL-TIME ANALYSIS
   - Operates as IDE sidecar with sub-second latency
   - Provides instant feedback as developers type
   - No waiting for builds or tests to fail

Why We Win:

Standard AI predicts the next word. FragilityGraph predicts the next break.

List of features offered by the solution

1. PRE-EMPTIVE IMPACT ANALYSIS

- Highlights downstream APIs affected by schema changes
- Detects data-type mismatches even if code compiles
- Shows cross-service dependencies automatically

2. REFACTOR CONFIDENCE SCORE

- Identifies "load-bearing" vs "modular" components
- Prioritizes high-risk refactoring areas
- Provides numerical confidence metrics (0-100%)

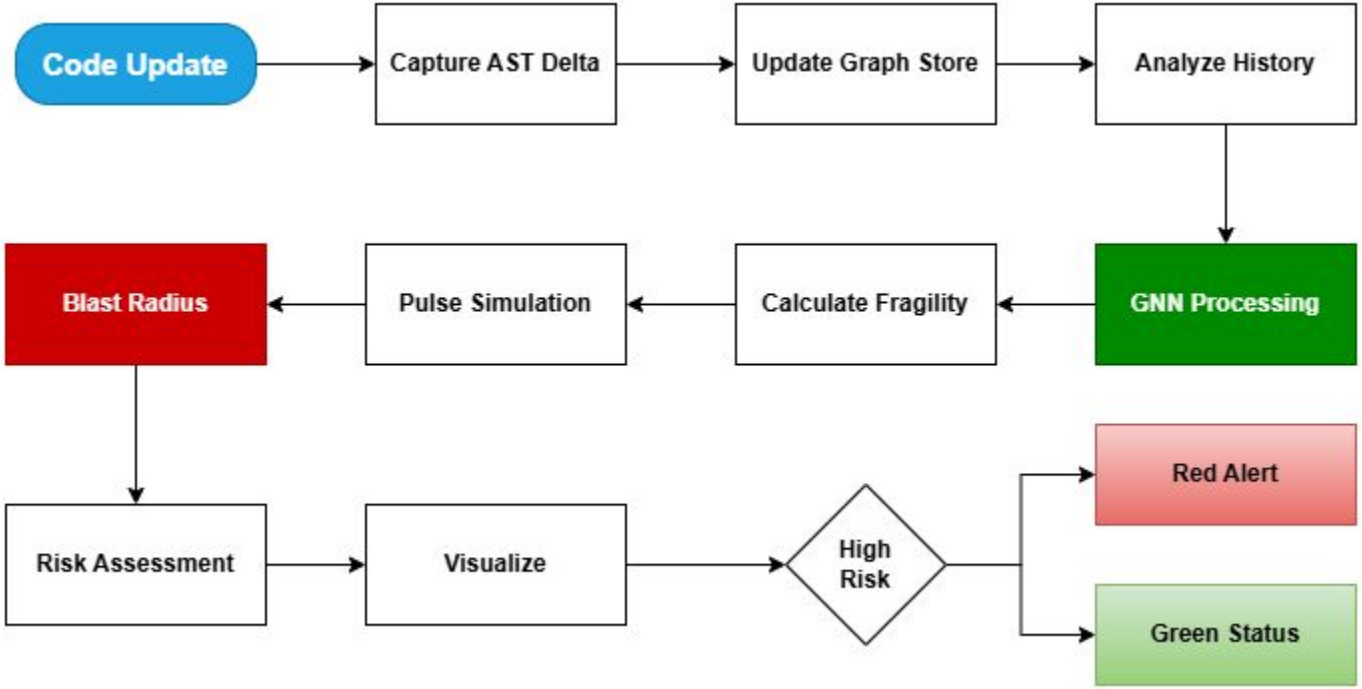
3. ONBOARDING DANGER ZONES

- Visual heatmap of codebase fragility
- Red zones indicate high-fragility areas
- Shows where small changes historically cause bugs
- Helps new team members navigate safely

4. BLAST RADIUS VISUALIZATION

- Real-time visual showing change propagation
- Displays affected microservices and modules
- Provides probability scores (e.g., 85% chance of regression)
- Interactive graph showing exact lines that will likely fail

Workflow diagram



Workflow description:

1. Developer makes a code change in IDE
2. Tree-sitter captures AST (Abstract Syntax Tree) delta in real-time
3. Graph store (Neo4j) updates with new nodes and edges
4. Git history analyzer identifies co-change patterns
5. GNN model processes the graph structure
6. System calculates fragility scores for each component
7. Pulse simulator injects stress signal at change point
8. Blast radius computation shows propagation scope
9. Risk assessment generates probability scores
10. Results visualized in IDE sidebar
11. High-risk changes trigger red alerts with details
12. Low-risk changes show green status
13. Developer can make informed decision

Architecture of the proposed solution:

LAYER 1: CLIENT LAYER
- IDE Extension (VS Code / JetBrains)
- User interface and interaction

LAYER 2: SENSOR LAYER
- Tree-sitter Parser (incremental AST extraction)
- AST Delta Extractor (tracks changes)

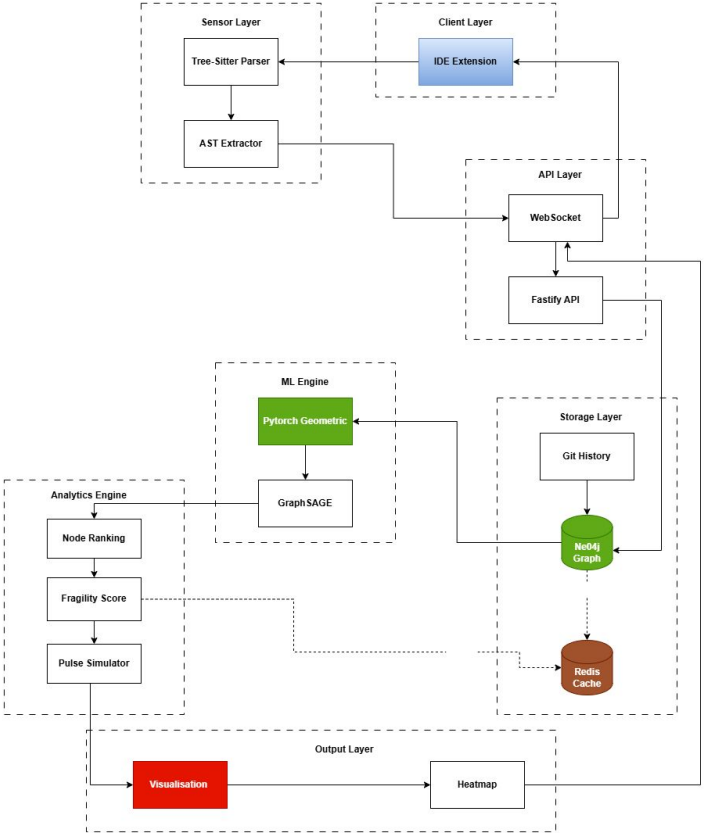
LAYER 3: API LAYER
- Fastify API Server (Node.js high-speed bridge)
- WebSocket Server (real-time bidirectional communication)

LAYER 4: STORAGE LAYER
- Neo4j Graph Database (nodes = classes, edges = dependencies)
- Redis Cache (fast lookups, model predictions)
- Git History Integration (temporal analysis)

LAYER 5: ML ENGINE
- PyTorch Geometric (graph neural network framework)
- GraphSAGE Model (node embedding and learning)

LAYER 6: ANALYSIS ENGINE
- Node Importance Ranking (identifies critical components)
- Fragility Score Calculator (computes risk metrics)
- Pulse Simulator (propagation modeling)

LAYER 7: OUTPUT LAYER
- Visualization Generator (creates visual representations)
- Heatmap Renderer (shows risk distribution)



Technologies to be used in the solution:

FRONTEND & IDE INTEGRATION:
- Tree-sitter (AST parsing and incremental updates)
- VS Code Extension API (editor integration)
- TypeScript/JavaScript (extension development)

BACKEND & API:
- Fastify (Node.js high-performance web framework)
- Python FastAPI (ML service endpoints)
- WebSocket (real-time bidirectional communication)

MACHINE LEARNING:
- PyTorch Geometric (graph neural network library)
- GraphSAGE / Relational GCN (GNN architectures)
- scikit-learn (evaluation metrics and preprocessing)

DATA & GRAPH STORAGE:
- Neo4j (graph database for code relationships)
- Redis (in-memory caching layer)
- Git integration (version control history mining)

FragilityGraph

Making Code Safer, One Graph at a Time

Mission:
- Predict breaks before they happen
- Refactor with confidence
- Code without fear

Call to Action:
Transform your development workflow from reactive debugging to proactive prediction.

Innovation partner H2S HACK2SKILL | Media partner YOURSTORY

AI for Bharat Hackathon

Powered by aws

Thank You